HOUSE PRICE PREDICTION - PROJECT CODE

Dipti Patil (57), Ishwari Mane (52), Akshata Mishra (53), Manali Madhavi (48), Poorva Kharade (43)

File: house_price_prediction.py

```
"""
House Price Prediction - Regression
===================================
Workflow: Dataset -> EDA -> Preprocessing -> Train/Test Split
          -> ML Model -> Evaluation -> Results -> Conclusion

Dataset : California Housing (1990 U.S. Census), loaded with
          sklearn.datasets.fetch_california_housing
          Source: Pace, R. K. & Barry, R. (1997), "Sparse Spatial Autoregressions",
          Statistics & Probability Letters. Available via scikit-learn / StatLib.
Target  : MedHouseVal  (median house value, in units of $100,000)

Run:  python house_price_prediction.py
Outputs are saved in the 'outputs/' folder (graphs + results_summary.md).
"""

import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")  # save figures without needing a display
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

RANDOM_STATE = 42
OUT_DIR = "outputs"
TARGET = "MedHouseVal"
os.makedirs(OUT_DIR, exist_ok=True)
sns.set_theme(style="whitegrid")


# ---------------------------------------------------------------------
# 1. DATASET
# ---------------------------------------------------------------------
def load_data() -> pd.DataFrame:
    """Load California Housing via scikit-learn (needs internet on first run).
    If that fails, fall back to a local file data/housing.csv with the same
    columns (MedInc, HouseAge, AveRooms, AveBedrms, Population, AveOccup,
    Latitude, Longitude, MedHouseVal)."""
    try:
        from sklearn.datasets import fetch_california_housing
        return fetch_california_housing(as_frame=True).frame
    except Exception as e:  # pragma: no cover
        local = os.path.join("data", "housing.csv")
        if os.path.exists(local):
            print(f"[info] sklearn download failed ({e}); using {local}")
            return pd.read_csv(local)
        raise SystemExit(
            "Could not download the dataset and data/housing.csv was not found.\n"
            "Connect to the internet and re-run, or place a CSV with the "
            "California Housing columns at data/housing.csv."
        )


# ---------------------------------------------------------------------
# 2. EDA
# ---------------------------------------------------------------------
def run_eda(df: pd.DataFrame, report: list) -> None:
    report.append("## Exploratory Data Analysis\n")
    report.append(f"- Shape: **{df.shape[0]} rows x {df.shape[1]} columns**")

```

```
    missing = df.isnull().sum()
    n_dup = int(df.duplicated().sum())
    report.append(f"- Missing values (total): **{int(missing.sum())}**")
    report.append(f"- Duplicate rows: **{n_dup}**\n")

    print("\n--- First 5 rows ---\n", df.head())
    print("\n--- Info ---")
    df.info()
    print("\n--- Summary statistics ---\n", df.describe().T)
    print("\n--- Missing values per column ---\n", missing)
    print("\nDuplicate rows:", n_dup)

    report.append("Summary statistics:\n")
    report.append(df.describe().T.round(3).to_markdown() + "\n")

    # Graph 1: distribution of the target variable
    plt.figure(figsize=(7, 4))
    sns.histplot(df[TARGET], bins=50, kde=True, color="#3b6ea5")
    plt.title("Distribution of Median House Value (target)")
    plt.xlabel("MedHouseVal (x $100,000)")
    plt.tight_layout()
    plt.savefig(f"{OUT_DIR}/1_target_distribution.png", dpi=150)
    plt.close()

    # Graph 2: correlation heatmap
    corr = df.corr()
    plt.figure(figsize=(9, 7))
    sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", center=0)
    plt.title("Correlation Heatmap")
    plt.tight_layout()
    plt.savefig(f"{OUT_DIR}/2_correlation_heatmap.png", dpi=150)
    plt.close()

    # Graph 3: strongest predictor vs target
    top_feat = corr[TARGET].drop(TARGET).abs().idxmax()
    plt.figure(figsize=(7, 5))
    plt.hexbin(df[top_feat], df[TARGET], gridsize=45, cmap="viridis", mincnt=1)
    plt.colorbar(label="Number of districts")
    plt.xlabel(top_feat)
    plt.ylabel("MedHouseVal (x $100,000)")
    plt.title(f"{top_feat} vs House Value")
    plt.tight_layout()
    plt.savefig(f"{OUT_DIR}/3_top_feature_vs_price.png", dpi=150)
    plt.close()

    capped = (df[TARGET] >= df[TARGET].max()).mean() * 100
    report.append("**Observations (computed from the data):**\n")
    report.append(f"- Target mean = {df[TARGET].mean():.2f}, median = "
                  f"{df[TARGET].median():.2f}, max = {df[TARGET].max():.2f}; "
                  f"skewness = {df[TARGET].skew():.2f} (right-skewed).")
    report.append(f"- About {capped:.1f}% of districts sit exactly at the maximum "
                  "value, which suggests the target is capped/censored.")
    report.append("- Correlation with the target (strongest first): "
                  + ", ".join(f"{k} ({v:+.2f})" for k, v in
                              corr[TARGET].drop(TARGET)
                              .sort_values(key=abs, ascending=False)
                              .head(4).items()) + ".")
    report.append(f"- Strongest single predictor: **{top_feat}**.\n")
    return top_feat


# ---------------------------------------------------------------------
# 3. PREPROCESSING helpers
# ---------------------------------------------------------------------
def add_features(df: pd.DataFrame) -> pd.DataFrame:
    """Simple, leakage-free feature engineering."""
    df = df.copy()
    df["BedroomRatio"] = df["AveBedrms"] / df["AveRooms"]
    return df


def main():
    report = ["# Results Summary (auto-generated by house_price_prediction.py)\n"]

    # ---- Dataset ----
    df = load_data()
```

```
    report.append("## Dataset\n")
    report.append("- California Housing (1990 US Census), via "
                  "`sklearn.datasets.fetch_california_housing`.")
    report.append(f"- Target variable: `{TARGET}` (median house value in $100,000s).")
    report.append(f"- Features: {', '.join(c for c in df.columns if c != TARGET)}.\n")

    # ---- EDA ----
    run_eda(df, report)

    # ---- Preprocessing ----
    report.append("## Preprocessing\n")
    df = df.drop_duplicates().reset_index(drop=True)
    df = add_features(df)
    X = df.drop(columns=[TARGET])
    y = df[TARGET]
    report.append("- Removed duplicate rows (if any).")
    report.append("- Added feature `BedroomRatio` = AveBedrms / AveRooms.")
    report.append("- Missing values: median imputation (fitted on training data only).")
    report.append("- Outliers: clipped to the 1st-99th percentile of the training data.")
    report.append("- Scaling: StandardScaler (used for Linear Regression).\n")

    # ---- Train / Test split (80/20) ----
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=RANDOM_STATE
    )
    report.append("## Train/Test Split\n")
    report.append(f"- 80% train ({len(X_train)} rows) / 20% test ({len(X_test)} rows), "
                  f"random_state={RANDOM_STATE}.\n")

    # Clip outliers using TRAIN percentiles only (avoids data leakage)
    low, high = X_train.quantile(0.01), X_train.quantile(0.99)
    X_train = X_train.clip(low, high, axis=1)
    X_test = X_test.clip(low, high, axis=1)

    # ---- Models ----
    models = {
        "Linear Regression (baseline)": Pipeline([
            ("impute", SimpleImputer(strategy="median")),
            ("scale", StandardScaler()),
            ("model", LinearRegression()),
        ]),
        "Random Forest Regressor": Pipeline([
            ("impute", SimpleImputer(strategy="median")),
            ("model", RandomForestRegressor(
                n_estimators=200, max_depth=None, min_samples_leaf=2,
                n_jobs=-1, random_state=RANDOM_STATE)),
        ]),
    }

    report.append("## Model Training & Evaluation\n")
    report.append("Metrics are computed on the **unseen test set**. "
                  "CV R2 = 5-fold cross-validation on the training set.\n")
    rows, preds = [], {}
    for name, pipe in models.items():
        print(f"\nTraining: {name}")
        pipe.fit(X_train, y_train)
        y_pred = pipe.predict(X_test)
        preds[name] = y_pred
        cv = cross_val_score(pipe, X_train, y_train, cv=5, scoring="r2", n_jobs=-1)
        rows.append({
            "Model": name,
            "MAE": mean_absolute_error(y_test, y_pred),
            "RMSE": np.sqrt(mean_squared_error(y_test, y_pred)),
            "R2": r2_score(y_test, y_pred),
            "CV R2 (mean)": cv.mean(),
        })
    res = pd.DataFrame(rows).set_index("Model").round(4)
    print("\n=== Test-set results ===\n", res)
    report.append(res.to_markdown() + "\n")

    best = res["R2"].idxmax()
    best_pipe, best_pred = models[best], preds[best]

    # Graph 4: actual vs predicted
    plt.figure(figsize=(6, 6))
    plt.scatter(y_test, best_pred, s=8, alpha=0.4, color="#3b6ea5")
```

```
    lims = [min(y_test.min(), best_pred.min()), max(y_test.max(), best_pred.max())]
    plt.plot(lims, lims, "r--", label="Perfect prediction")
    plt.xlabel("Actual (x $100,000)")
    plt.ylabel("Predicted (x $100,000)")
    plt.title(f"Actual vs Predicted - {best}")
    plt.legend()
    plt.tight_layout()
    plt.savefig(f"{OUT_DIR}/4_actual_vs_predicted.png", dpi=150)
    plt.close()

    # Graph 5: feature importance (Random Forest)
    rf = models["Random Forest Regressor"].named_steps["model"]
    imp = pd.Series(rf.feature_importances_, index=X_train.columns).sort_values()
    plt.figure(figsize=(7, 5))
    imp.plot(kind="barh", color="#3b6ea5")
    plt.title("Random Forest - Feature Importance")
    plt.tight_layout()
    plt.savefig(f"{OUT_DIR}/5_feature_importance.png", dpi=150)
    plt.close()

    # ---- Results & Conclusion ----
    b = res.loc[best]
    report.append("## Results\n")
    report.append(f"- Best model: **{best}** with R2 = {b['R2']:.3f}, "
                  f"MAE = {b['MAE']:.3f} (about ${b['MAE'] * 100000:,.0f}), "
                  f"RMSE = {b['RMSE']:.3f} (about ${b['RMSE'] * 100000:,.0f}).")
    top3 = imp.sort_values(ascending=False).head(3)
    report.append("- Most important features: "
                  + ", ".join(f"{k} ({v:.2f})" for k, v in top3.items()) + ".")
    lin_r2 = res.loc["Linear Regression (baseline)", "R2"]
    rf_r2 = res.loc["Random Forest Regressor", "R2"]
    report.append(f"- Random Forest vs Linear Regression R2: {rf_r2:.3f} vs {lin_r2:.3f}.\n")
    report.append("## Conclusion\n")
    report.append(
        f"The {best} explains about {b['R2'] * 100:.1f}% of the variance in house "
        "values on unseen data. Comparing it with the linear baseline shows how much "
        "non-linear relationships matter in this problem. Limitations: the data are "
        "from 1990, the target is capped at its maximum, and predictions are for "
        "district-level medians, not individual houses. Future work: hyperparameter "
        "tuning (GridSearchCV), gradient boosting, and location-based features.\n")
    report.append("## Dataset Source\n")
    report.append("Pace, R. K. and Barry, R. (1997). Sparse Spatial Autoregressions. "
                  "Statistics & Probability Letters, 33, 291-297. Accessed through "
                  "scikit-learn: https://scikit-learn.org/stable/modules/generated/"
                  "sklearn.datasets.fetch_california_housing.html")

    with open(f"{OUT_DIR}/results_summary.md", "w", encoding="utf-8") as f:
        f.write("\n".join(report))
    print(f"\nDone. Graphs and results_summary.md saved in '{OUT_DIR}/'.")


if __name__ == "__main__":
    main()
```

File: requirements.txt

```
numpy
pandas
matplotlib
seaborn
scikit-learn
tabulate
```